

Regrex Cheatsheet

[0 - Index](#)

il doppio backslash `\\` . Poiché il backslash è già un carattere di escape per le stringhe in Java (es. `\n` per andare a capo), per scrivere una vera regex devi "scappare" il backslash stesso. Quindi, la regex `\d` in Java si scrive sempre `\\d` .

Classi di caratteri (Cosa cerco?):

- `\\d` : Un numero qualsiasi (da 0 a 9).
- `\\D` : Qualsiasi cosa *tranne* un numero.
- `\\s` : Uno spazio bianco (spazio, tab, a capo).
- `\\S` : Qualsiasi cosa *tranne* uno spazio bianco.
- `\\w` : Un carattere alfanumerico (lettere, numeri o underscore `_`).
- `\\W` : Un carattere non alfanumerico (es. punteggiatura).
- `.` : Il carattere "jolly". Vale per *qualsiasi* carattere (tranne l'a capo).

*Quantificatori (Quante volte lo cerco?):

- `+` : Una o più volte. (Es. `\\d+` = almeno un numero, ma anche mille numeri consecutivi).
- `*` : Zero o più volte. (Può anche non esserci).
- `?` : Zero o una volta. (Rende l'elemento *opzionale*).
- `{n}` : Esattamente *n* volte. (Es. `\\d{3}` = esattamente 3 numeri).
- `{n,m}` : Da *n* a *m* volte. (Es. `\\w{2,4}` = da 2 a 4 lettere/numeri).

Insiemi e Gruppi (Logica avanzata):

- `[abc]` : Solo 'a', 'b', o 'c'.
- `[a-z]` : Tutte le lettere minuscole dalla 'a' alla 'z'.
- `[^0-9]` : Il cappelletto `^` dentro le quadre significa **NOT**. Qui significa: tutto *tranne* i numeri da 0 a 9.
- `(abc)` : Raggruppa i caratteri nella sequenza esatta "abc".
- `A|B` : Operatore OR. Cerca il pattern A oppure il pattern B.

Ancore (Dove lo cerco?):

- `^` : Inizio della stringa (se usato fuori dalle parentesi quadre).
- `$` : Fine della stringa.

Danger

Cercando di splittare con uno qualsiasi dei caratteri speciali per utilizzarlo come carattere e non come elemento logico devo precederlo con `\\`

i Metodi di Java per le Regex

Java ti offre tre metodi comodissimi direttamente nella classe `String` :

- `stringa.matches(regex)` : Restituisce `true` se l'INTERA stringa corrisponde perfettamente al pattern.
- `stringa.split(regex)` : **Taglia la stringa in un array, usando il pattern come "forbice"**. rimuove i caratteri forbice.
- `stringa.replaceAll(regex, sostituto)` : Trova tutte le parti che combaciano con il pattern e le sostituisce con un'altra stringa.

```
1 // Abbiamo una riga di un file CSV dove i campi sono separati da virgola
2 // MA a volte c'è uno spazio dopo la virgola, a volte no.
3 String rigaCsv = "Marco, Rossi, 25,Roma, Studente";
4
5 // Tagliamo dove c'è una virgola, seguita OPZIONALMENTE (*) da spazi bianchi (\\s)
6 String[] campi = rigaCsv.split(",\\s*");
7
8 for (String campo : campi) {
9     System.out.println("-" + campo + "-");
10 }
11 // Output pulitissimo, senza spazi iniziali/finali nei campi:
12 // -Marco-
13 // -Rossi-
14 // -25-
15 // -Roma-
16 // -Studente-
```